

# Textual Data Analysis with NLTK

## Objectives

- ✓ NLTK Library
- ✓ Search for a Word
- ✓ Analyze the Frequency of Words
- ✓ Select Words from Text
- ✓ Bigrams and Collocations
- ✓ Sentiment Analysis

# Text Analysis in Data Analytics

- **Text Analysis in Data Analytics** is the process of converting **unstructured textual data** (such as emails, reviews, social media posts, reports, or transcripts) into **structured, measurable insights** using computational and statistical techniques.
- Text analysis allows us to:
  - Quantify opinions
  - Detect trends
  - Identify key topics
  - Classify documents
  - Predict sentiment

- In modern organizations, a large portion of data is **textual**, including:
  - Customer reviews
  - Social media comments
  - Support tickets
  - News articles
  - Emails
  - Survey responses
- Unlike numerical data, text is **unstructured**, meaning it cannot be directly analyzed using traditional statistical methods.

Text analysis typically involves three main stages:

- **1. Text Preprocessing (Cleaning)**
  - Purpose: Prepare raw text for analysis.
  - Includes:
    - Lowercasing
    - Removing punctuation
    - Removing stopwords
    - Tokenization
    - Stemming or lemmatization

## • 2. Feature Extraction

- Convert text into numerical representations.
- Common methods:
  - Bag of Words (BoW)
  - TF (Term Frequency)
  - TF-IDF
  - N-grams
- Text → Numbers → Machine Learning

### • **3. Modeling & Interpretation**

- Apply analytical methods such as:
- Sentiment Analysis
- Topic Modeling
- Text Classification
- Clustering
- Named Entity Recognition

## Applications in Data Analytics

- **Business Analytics**
  - Analyze customer feedback
  - Brand sentiment monitoring
  - Product review mining
- **Marketing Analytics**
  - Social media sentiment tracking
  - Campaign performance analysis
- **Financial Analytics**
  - News sentiment analysis
  - Risk detection
- **Healthcare Analytics**
  - Clinical note analysis
  - Patient feedback mining

- Example
  - Raw Text:
  - "The service was slow but the staff was very friendly."
- After Text Analysis:
  - Sentiment: Mixed (Positive & Negative)
  - Keywords: service, slow, staff, friendly
  - Category: Customer Feedback

# Natural Language Toolkit

**NLTK** is:

- An open-source Python library
- Designed for teaching and research in NLP
- Provides easy access to:
  - Text processing tools
  - Linguistic resources
  - Corpora and lexical databases
- It is commonly used in:
  - Data analytics
  - Machine learning preprocessing
  - Academic NLP research
  - Text mining

- **Major Text Analysis Techniques Using NLTK**
- Below are the core techniques used in text analytics with NLTK:
- **1. Tokenization**
  - Breaking text into smaller units.
    - **Sentence Tokenization** → split into sentences
    - **Word Tokenization** → split into words
- Example:

"I love data analytics."

→ ["I", "love", "data", "analytics"]
- Purpose:
  - Prepares text for further analysis

## • **2. Stopword Removal**

- Removing common words that carry little meaning.
- Examples:
  - the
  - is
  - and
  - in
- Purpose:
  - Reduces noise
  - Improves model performance

- **3. Stemming**

- Reducing words to their root form by cutting suffixes.

- Example:

- running → run
- studies → studi

- Common NLTK stemmer:

- PorterStemmer

- **4. Lemmatization**

- Reducing words to their dictionary base form.

- Example:

- running → run
- better → good

- More linguistically accurate than stemming.

- **5. Part-of-Speech (POS) Tagging**
- Identifying grammatical roles of words.
- Example:
  - run → Verb
  - data → Noun
- Used for:
  - Syntax analysis
  - Information extraction

- **6. Named Entity Recognition (NER)**
- Identifying real-world entities:
  - Person names
  - Organizations
  - Locations
  - Dates
- Example:
  - "Google was founded in California."
  - Google (Organization)
  - California (Location)

- **7. Frequency Distribution**

- Counting word occurrences.
- Used for:
  - Keyword extraction
  - Text summarization
  - Trend analysis

- **8. N-grams Analysis**

- Analyzing word combinations.
  - Unigram → single words
  - Bigram → 2-word combinations
  - Trigram → 3-word combinations
- Example:
  - "I love data science"
  - Bigram: (I love), (love data), (data science)

- **9. Sentiment Analysis (Basic)**
- Using lexical approaches to classify text as:
  - Positive
  - Negative
  - Neutral
- Often combined with:
  - VADER sentiment analyzer in NLTK
- **10. Text Classification**
- Using:
  - Naïve Bayes
  - Decision Trees
  - Other ML models
- For tasks like:
  - Spam detection
  - Topic classification
  - Document categorization

## • **Typical Text Analysis Workflow in NLTK**

- Load text data
- Clean text (remove punctuation, lowercase)
- Tokenize
- Remove stopwords
- Apply stemming/lemmatization
- Feature extraction (frequency, n-grams)
- Apply ML model or analysis

- **Advantages:**

- Excellent for teaching
- Large built-in corpora
- Easy to understand API
- Good for prototyping NLP models

- **Limitations:**

- Slower than modern libraries like spaCy
- More academic than production-focused

# Import the NLTK Library

```
import nltk
```

```
nltk.download_shell()
```

NLTK Downloader

d) Download l) List u) Update c) Config h) Help q) Quit

Downloader: **l**

Packages:

```
[ ] abc..... Australian Broadcasting Commission 2006
[ ] alpino..... Alpino Dutch Treebank
[ ] averaged_perceptron_tagger Averaged Perceptron Tagger
[ ] averaged_perceptron_tagger_eng Averaged Perceptron Tagger (JSON)
[ ] averaged_perceptron_tagger_ru Averaged Perceptron Tagger (Russian)
[ ] averaged_perceptron_tagger_rus Averaged Perceptron Tagger (Russian)
[ ] basque_grammars..... Grammars for Basque
[ ] bcp47..... BCP-47 Language Tags
[ ] biocreative_ppi..... BioCreAtIvE (Critical Assessment of Information
    Extraction Systems in Biology)
[ ] bllip_wsj_no_aux.... BLLIP Parser: WSJ Model
[ ] book_grammars..... Grammars from NLTK Book
[ ] brown..... Brown Corpus
[ ] brown_tei..... Brown Corpus (TEI XML Version)
[ ] cess_cat..... CESS-CAT Treebank
[ ] cess_esp..... CESS-ESP Treebank
[ ] chat80..... Chat-80 Data Files
[ ] city_database..... City Database
[ ] cmudict..... The Carnegie Mellon Pronouncing Dictionary (0.6)
[ ] comparative_sentences Comparative Sentence Dataset
```

Hit Enter to continue: **d**

```
[ ] comtrans..... ComTrans Corpus Sample
[ ] conll2000..... CONLL 2000 Chunking Corpus
[ ] conll2002..... CONLL 2002 Named Entity Recognition Corpus
[ ] conll2007..... Dependency Treebanks from CoNLL 2007 (Catalan
    and Basque Subset)
[ ] crubadan..... Crubadan Corpus
[ ] dependency_treebank. Dependency Parsed Treebank
[ ] dolch..... Dolch Word List
[ ] english_wordnet..... Open English Wordnet
[ ] europarl_raw..... Sample European Parliament Proceedings Parallel
    Corpus
[ ] extended_omw..... Extended Open Multilingual WordNet
[ ] floresta..... Portuguese Treebank
[ ] framenet_v15..... FrameNet 1.5
[ ] framenet_v17..... FrameNet 1.7
[ ] gazetteers..... Gazeteer Lists
[ ] genesis..... Genesis Corpus
[ ] gutenberg..... Project Gutenberg Selections
[ ] ieer..... NIST IE-ER DATA SAMPLE
[ ] inaugural..... C-Span Inaugural Address Corpus
[ ] indian..... Indian Language POS-Tagged Corpus
```

Hit Enter to continue: q

d) Download l) List u) Update c) Config h) Help q) Quit

Downloader: **q**

- When you already know the name of the package you want to download, just enter the command `nltk.download()` with the package name as an argument.

```
nltk.download('gutenberg')

[nltk_data] Downloading package gutenberg to /root/nltk_data...
[nltk_data] Unzipping corpora/gutenberg.zip.
True
```

- see the contents of the package thanks to the **fileids()** function

```
gb = nltk.corpus.gutenberg
print("Gutenberg files : ", gb.fileids())

Gutenberg files : ['austen-emma.txt', 'austen-persuasion.txt', 'austen-sense.txt', 'bible-kjv.txt', 'blake-poems.txt', 'bryant-stories
```

**Gutenberg files** are a package consisting of many text files: 'austen-emma.txt', 'austen-persuasion.txt', 'austen-sense.txt', 'biblekjv.txt', 'blake-poems.txt', 'bryant-stories.txt', 'burgess-busterbrown.txt', 'carrollalice.txt', 'chesterton-ball.txt', 'chesterton-brown.txt', 'chesterton-thursday.txt', 'edgeworth-parents.txt', 'melville-moby\_dick.txt', 'milton-paradise.txt', 'shakespearecaesar.txt', 'shakespeare-hamlet.txt', '**shakespeare-macbeth.txt**', 'whitman-leaves.txt'

- To access the internal content of one of these files, you first select one, for example Shakespeare's Macbeth (shakespeare-macbeth.txt), and then assign it to a variable of convenience.
- An extraction mode is for words, that is, you want to create an array containing words as elements. In this regard, you need to use the **words()** function.

```
macbeth = nltk.corpus.gutenberg.words('shakespeare-macbeth.txt')
macbeth

['[', 'The', 'Tragedie', 'of', 'Macbeth', 'by', ...]
```

- See the length of this text (in words): the **len()** function.

```
len(macbeth)

23140
```

The text used for these examples is therefore composed of 23140 words.

- The macbeth variable is a long array containing the words of the text. If you want to see the first ten words of the text:

```
macbeth[:10]
```

```
[['',  
  'The',  
  'Tragedie',  
  'of',  
  'Macbeth',  
  'by',  
  'William',  
  'Shakespeare',  
  '1603',  
  '']]
```

the first ten words contain the title of the work, but also the square brackets, which indicate the beginning and end of a sentence.

- Used the sentence extraction mode with the `sents()` function, you would have obtained a more structured array, with each sentence as an element. These elements, in turn, would be arrays with words for elements.

```
nltk.download('punkt_tab')
macbeth_sents = nltk.corpus.gutenberg.sents('shakespeare-macbeth.txt')
macbeth_sents[:5]
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[['[',
  'The',
  'Tragedie',
  'of',
  'Macbeth',
  'by',
  'William',
  'Shakespeare',
  '1603',
  ']''],
 ['Actus', 'Primus', '.'],
 ['Scoena', 'Prima', '.'],
 ['Thunder', 'and', 'Lightning', '.'],
 ['Enter', 'three', 'Witches', '.']]
```

## Search for a Word with NLTK

- When working with an **NLTK corpus** (a collection of words extracted from text), one of the most fundamental tasks is **searching within the corpus**.
- In NLTK, searching is typically done using the **concordance()** function.
- The `concordance(word)` function finds **all occurrences of a specific word** in the corpus and displays them with surrounding context.
- The **first time** you run `concordance()`, it may take several seconds because NLTK creates an **index of the corpus** to speed up searching.
- After this index is built, **subsequent searches are much faster** since the system reuses the existing index.
  - Before using `concordance()`, ensure that the corpus is an object of type `nltk.Text`.

- make sure that the corpus is an object `nltk.Text`, and then search internally for the word 'Stage'.

```
text = nltk.Text(macbeth)
```

```
text.concordance('Stage')
```

Displaying 3 of 3 matches:

nts with Dishes and Service ouer the Stage . Then enter Macbeth Macb . If it we  
with mans Act , Threatens his bloody Stage : byth ' Clock ' tis Day , And yet d  
struts and frets his houre vpon the Stage , And then is heard no more . It is

- You have obtained three different occurrences of the text.
  - Another form of searching for a word present in NLTK is that of context. That is, the previous word and the word next to the one you are looking for: use the **common\_contexts()** function.

```
text.common_contexts(['Stage'])
```

```
the_. bloody_: the_.
```

- If you look at the results of the previous research, you can see that the three results correspond to what has been said.
  - Once you understand how NLTK conceives the concept of the word and its context during the search, it is easy to understand the concept of a synonym. That is, it is assumed that all words that have the same context can be possible synonyms. To search for all words that have the same context as the searched one: use the **similar()** function.

```
text.similar('Stage')
```

```
day time face warre ayre king bleeding man reuolt serieant like  
knowledge broyle shew head spring heeles hare thane skie
```

## Analyze the Frequency of Words

- One of the most fundamental techniques in text analysis is calculating the **frequency of words** in a text. Word frequency analysis helps identify:
  - The most common terms
  - Important themes or topics
  - Patterns in language usage
- Because this operation is so common, **NLTK** provides a built-in function called `FreqDist()`.
- **`nltk.FreqDist()`**
  - It calculates how many times each word appears in a corpus.
  - It takes a **list (array) of words** as its argument.
  - It returns a frequency distribution object that behaves like a dictionary (word → count).

- Get a statistical distribution of all the words in the text.

```
fd = nltk.FreqDist(macbeth)
```

- see the first ten most common words in the text, you can use the **most\_common()** function.

```
fd.most_common(10)
```

```
[(',', 1962),  
 ('.', 1235),  
 ('"', 637),  
 ('the', 531),  
 (':', 477),  
 ('and', 376),  
 ('I', 333),  
 ('of', 315),  
 ('to', 311),  
 ('?', 241)]
```

- Stopwords are words that have little meaning in the analysis and must be filtered. There is no general rule to determine whether a word is a stopword (to be deleted) or not.
- The NLTK library comes to the rescue by providing you with an array of pre-selected stopwords. To download these stopwords, you can use the **nltk.download()** command.

```
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...  
[nltk_data]   Unzipping corpora/stopwords.zip.  
True
```

- Can select only those related to English, saving them in a variable sw.

```
sw = set(nltk.corpus.stopwords.words('english'))  
print(len(sw))  
list(sw)[:10]
```

```
198  
['the',  
 "wouldn't",  
 'over',  
 "you'd",  
 'has',  
 "it'll",  
 'to',  
 'mightn',  
 'my',  
 'shan']
```

There are **198** stopwords in the English vocabulary according to NLTK.

- Use these stopwords to filter the **macbeth** variable.

```
macbeth_filtered = [w for w in macbeth if w.lower() not in sw]
```

```
len(macbeth_filtered)
```

```
14946
```

```
fd = nltk.FreqDist(macbeth_filtered)
fd.most_common(10)
```

```
[(',', 1962),
 ('.', 1235),
 ('"', 637),
 (':', 477),
 ('?', 241),
 ('Macb', 137),
 ('haue', 117),
 ('-', 100),
 ('Enter', 80),
 ('thou', 63)]
```

Now that the first ten most common words are returned, you can see that the stopwords have been eliminated, but the result is still not satisfactory.

- In fact, punctuation is still present in the words. To eliminate all punctuation, you can change the previous code by inserting in the filter an array of punctuation containing the punctuation symbols.
- This punctuation array can be obtained by importing the **string** function.

```
import string
punctuation = set(string.punctuation)
macbeth_filtered2 = [w.lower() for w in macbeth if w.lower()
    not in sw and w.lower() not in punctuation]
```

Recalculate the frequency distribution of words.

```
fd = nltk.FreqDist(macbeth_filtered2)
fd.most_common(10)

[('macb', 137),
 ('haue', 122),
 ('thou', 90),
 ('enter', 81),
 ('shall', 68),
 ('macbeth', 62),
 ('vpon', 62),
 ('thee', 61),
 ('macd', 58),
 ('vs', 57)]
```

## Select Words from Text

- Another form of processing and data analysis is the process of selecting words contained in a body of text based on particular characteristics. For example, you might be interested in extracting words based on their length.
- To get all the longest words, for example words that are longer than 12 characters, you enter the following command.

```
long_words = [w for w in macbeth if len(w)> 12]
```

- List them in alphabetical order by using the **sort()** function.

```
sorted(long_words)
```

```
['Assassination',  
'Chamberlaines',  
'Distinguishes',  
'Gallowgrosses',  
'Metaphysicall',  
'Northumberland',  
'Voluptuousnesse',  
'commendations',  
'multitudinous',  
'supernaturall',  
'vnaccompanied']
```

- To look for all the words that contain a certain sequence of characters, such as 'ious': change the condition in the for in loop to get the desired selection.

```
ious_words = [w for w in macbeth if 'ious' in w]
ious_words = set(ious_words)
sorted(ious_words)
```

```
['Avaricious',
'Gracious',
'Industrious',
'Iudicious',
'Luxurious',
'Malicious',
'Oblivious',
'Pious',
'Rebellious',
'compunctious',
'furious',
'gracious',
'pernicious',
'pernitious',
'pious',
'precious',
'rebellious',
'sacrilegious',
'serious',
'spacious',
'tedious']
```

## Bigrams and Collocations

- Another basic element of text analysis is to consider pairs of words (**bigrams**) instead of single words.
  - The words “is” and “yellow” are for example a bigram, since their combination is possible and meaningful. So “is yellow” can be found in textual data.
- We all know that some of these bigrams are so common in our literature that they are almost always used together.
  - Examples include “fast food,” “pay attention,” “good morning,” and so on. These bigrams are called **collocations**.

- Textual analysis can also involve the search for any bigrams within the text under examination.
- To find them, simply use the **bigrams()** function: In order to exclude stopwords and punctuation from the bigrams, you must use the set of words already filtered, such as `macbeth_filtered2`.

```
bgrms = nltk.FreqDist(nltk.bigrams(macbeth_filtered2))  
bgrms.most_common(15)
```

```
[(('enter', 'macbeth'), 16),  
 (('exeunt', 'scena'), 15),  
 (('thane', 'cawdor'), 13),  
 (('knock', 'knock'), 10),  
 (('st', 'thou'), 9),  
 (('thou', 'art'), 9),  
 (('lord', 'macb'), 9),  
 (('haue', 'done'), 8),  
 (('macb', 'haue'), 8),  
 (('good', 'lord'), 8),  
 (('let', 'vs'), 7),  
 (('enter', 'lady'), 7),  
 (('wee', '1'), 7),  
 (('would', 'st'), 6),  
 (('macbeth', 'macb'), 6)]
```

- In addition to the bigrams, there can also be placements based on trigrams, which are combinations of three words.
  - The **trigrams()** function is used.

```
tgrms = nltk.FreqDist(nltk.trigrams (macbeth_filtered2))  
tgrms.most_common(10)
```

```
[('knock', 'knock', 'knock'), 6),  
 (('enter', 'macbeth', 'macb'), 5),  
 (('enter', 'three', 'witches'), 4),  
 (('exeunt', 'scena', 'secunda'), 4),  
 (('good', 'lord', 'macb'), 4),  
 (('three', 'witches', '1'), 3),  
 (('exeunt', 'scena', 'tertia'), 3),  
 (('thunder', 'enter', 'three'), 3),  
 (('exeunt', 'scena', 'quarta'), 3),  
 (('scena', 'prima', 'enter'), 3)]
```

## Preprocessing Steps

- **Text preprocessing** is one of the most essential and foundational stages in text analysis and NLP.
- After collecting text data from various sources (documents, websites, social media, etc.), the raw text is usually **not ready for analysis**. It must first be:
  - **Cleaned** (remove noise, symbols, unwanted characters)
  - **Transformed** (standardize format)
  - **Prepared** (structured for NLP techniques)
- These steps ensure the text can be effectively used in parsing, modeling, and other NLP tasks.

- **Lowercasing (Lower Conversion)**

- One of the most common preprocessing operations is **converting all text to lowercase**.
- Why?
  - Text often contains words with capital letters (e.g., *Word*, *WORD*, *word*).
  - Many NLP techniques treat words as distinct tokens.
  - Without lowercasing, "Word" and "word" would be considered different words.
- By converting everything to lowercase:
  - "Word" → "word"
  - "WORD" → "word"
- This ensures **consistency**, reduces vocabulary size, and improves analysis accuracy.

```
text = 'This is a Demo Sentence'
lower_text = text.lower()
lower_text

'this is a demo sentence'
```

- *Word tokenization* is another very common operation in NLP.
- Text consists of words with spaces between them. Therefore, the operation of converting text into a list of words is fundamental in order to be able to process the text computationally.
- NLTK provides the **word\_tokenize()** function for this purpose. But first you need to download the 'punkt' resource from nltk.

```
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...  
[nltk_data]   Unzipping tokenizers/punkt.zip.  
True
```

```
text = 'This is a Demo Sentence'  
tokens = nltk.word_tokenize(text)  
tokens
```

```
['This', 'is', 'a', 'Demo', 'Sentence']
```

- Tokenization can also be performed at a higher level, by separating the sentences that make up the text and converting them into elements of a list, instead of single words.
  - The **sent\_tokenize()** function.

```
text = 'This is a Demo Sentence. This is another sentence'  
tokens = nltk.sent_tokenize(text)  
tokens
```

```
['This is a Demo Sentence.', 'This is another sentence']
```

- Another important step in **text preprocessing** is **punctuation removal**.
- When analyzing text from sources such as:
  - Social media comments
  - Online product reviews
  - Web content

you will often encounter a large number of punctuation marks (e.g., !, ?, ..., ,, #, etc.).

- These punctuation marks can:
  - Interfere with proper **tokenization**
  - Create noisy or incorrect tokens
  - Reduce the accuracy of text analysis

- **Using RegexpTokenizer in NLTK**

To handle this, **NLTK** provides a flexible tokenizer called RegexpTokenizer.

- It allows you to define custom tokenization rules using **regular expressions (regex)**.
- You can specify patterns that extract only words and ignore punctuation.

```
from nltk.tokenize import RegexpTokenizer
```

```
text = 'This% is a #!!@ Sentence full of punctuation marks :-) '  
regext = RegexpTokenizer(r'[a-zA-Z0-9]+')  
tokens = regext.tokenize(text)  
tokens
```

```
['This', 'is', 'a', 'Sentence', 'full', 'of', 'punctuation', 'marks']
```

- **Stopword removal** is a more advanced preprocessing step in text analysis.
- Unlike punctuation (which can be removed using regular expressions), **stopwords are actual words**. They are very common words that:
  - Appear frequently
  - Carry little meaningful information
  - Do not significantly contribute to the overall topic of the text
- Examples in English include:  
*the – a – on - in*
- Stopwords are **language-specific**, meaning each language has its own list of stopwords.

```
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...  
[nltk_data] Package stopwords is already up-to-date!  
True
```

```
from nltk.corpus import stopwords
```

```
text = 'This is a Demo Sentence. This is another sentence'  
eng_sw = stopwords.words('english')  
tokens = nltk.word_tokenize(text)  
clean_tokens = [word for word in tokens if word not in eng_sw]  
clean_tokens
```

```
['This', 'Demo', 'Sentence', '.', 'This', 'another', 'sentence']
```

- Another important category of **text preprocessing** involves **linguistic normalization**, specifically:
  - **Stemming**
  - **Lemmatization**
- Both techniques reduce words to a common base form, but they work differently.

- **Stemming**

- Reduces words to their **root form** (stem).
- The root may not be a real dictionary word.
- Words with the same root are treated as the same token.

- Example:

- *playing, played, plays* → **play**
- *connection, connected, connecting* → **connect**

- Stemming is:

- Faster
- Rule-based
- Less linguistically precise

- **Using SnowballStemmer in NLTK**

```
from nltk.stem import SnowballStemmer
```

```
text = 'This operation operates for the operator curiosity. A decisive decision'  
stemmer = SnowballStemmer('english')  
tokens = nltk.word_tokenize(text)  
stemmed_tokens = [stemmer.stem(word) for word in tokens]  
print(stemmed_tokens)
```

```
['this', 'oper', 'oper', 'for', 'the', 'oper', 'curios', '.', 'a', 'decis', 'decis']
```

- **Lemmatization**

- Reduces words to their **lemma** (dictionary base form).
- Groups all inflected forms under a real word.
- More linguistically accurate than stemming.

- Example:

- *better* → **good**
- *was, were, is* → **be**
- *running* → **run**

- Lemmatization:

- Uses vocabulary and morphological analysis
- Is more precise
- Usually requires part-of-speech (POS) information

```
nltk.download('omw-1.4')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

```
from nltk.stem import WordNetLemmatizer
```

```
text = 'A verb: I split, it splits. Splitted verbs.'
tokens = nltk.word_tokenize(text)
lmtzr = WordNetLemmatizer()
lemma_tokens = [lmtzr.lemmatize(word) for word in tokens]
print(lemma_tokens)
```

```
['A', 'verb', ':', 'I', 'split', ',', 'it', 'split', '.', 'Splitted', 'verb', '.']
```

## Use Text on the Network

- To access the Internet to extract the text and collect it as a corpus to be used for analysis with NLTK.
- First, you need to import a library that allows you to connect to the contents of web pages. The **urllib** library is an excellent candidate for this purpose, as it allows you to download the text content from the Internet, including HTML pages.
- Import the **request()** function

```
from urllib import request
```

- Write the URL of the page that contains the text to be extracted.
- Still referring to the **gutenberg** project, you can choose,
  - Example: a book written by Dostoevsky ([www.gutenberg.org](http://www.gutenberg.org)).

```
url = "http://www.gutenberg.org/files/2554/2554-0.txt"
response = request.urlopen(url)
raw = response.read().decode('utf8')
```

- Check the contents: the first 75 characters.

```
new_raw = raw[:75]
new_raw
```

```
'*** START OF THE PROJECT GUTENBERG EBOOK 2554 ***\n\n\n\nCRIME AND PUNISHMENT\n'
```

- Convert it into a corpus compatible with NLTK.
- These commands do nothing more than split the character text into tokens (that is, words) using the **nltk.word\_tokenize()** function and then convert the tokens into a textual body suitable for NLTK using **nltk.Text()**.

```
tokens = nltk.word_tokenize (new_raw)
webtext = nltk.Text (tokens)
```

```
webtext[:12]
```

```
['*',
 '*',
 '*',
 '*',
 'START',
 'OF',
 'THE',
 'PROJECT',
 'GUTENBERG',
 'EBOOK',
 '2554',
 '*',
 '*']
```

# Homework

- Extract the Text from the HTML Pages
- Sentiment Analysis

